Lesson notes · for you

The bus route

Optimization · Level 3 · Lesson 3

Time
20 min

Materials
Paper and a pen; Rough distances between five places you know; Optional: a phone with Darsi to run your program

Learning goals

- Explain why the number of routes explodes.
- Write nearest neighbour in Python and improve a route with 2-opt.
- Judge a route by its length and its longest ride.

Activity: A fairer round trip

1. Map (5 min): Draw five places as dots and write a rough distance between every pair, in a table like d.
2. Build (5 min): From your starting place, always go to the closest place not yet visited, then back. Draw the route and add up its length.
3. Improve (5 min): If two roads cross, reverse the stretch between them and add up again. Repeat while it helps.
4. Ride (5 min): For each stop, add the distance from there to the end. Do it for both directions and choose the one with the shorter longest ride.

Distances can be minutes of walking. Use made-up names if you prefer not to show real places.

Discussion

- Why does nearest neighbour often end with a long leg?
- How can 2-opt stop before the best route?
- Should the bus ever skip a stop to save fuel?

Check yourself

- Draw a route with a crossing + the route after one 2-opt swap.

Try it this week
Map five places you visit in a week (home, school, the market, a friend's house, the clinic) with rough distances. Plan a round trip from home by nearest neighbour, draw it, and look for a crossing: swap it and add up the distance again. Then ask: if a friend joined at your first stop, how long would they ride?

Worksheet

The bus route

Optimization · Level 3 · Lesson 3

Name _____ Class _____ Date _____

1 **Two routes have the same length. On one a village rides 50 minutes; on the other nobody rides more than 30. Which should the school choose?**

- ◯ The one where nobody rides more than 30 minutes
- ◯ Either: they cost the same
- ◯ Whichever the driver prefers

2 **On a map where every road is a straight line, when two roads in a bus route cross each other, a 2-opt swap can always make the route shorter.**

- ◯ True
- ◯ False

3 **What does this program print?**

```
route = [0, 1, 2, 3, 4]
i, j = 1, 3
mid = list(reversed(route[i:j + 1]))
print(route[:i] + mid + route[j + 1:])
```

- ◯ [0, 3, 2, 1, 4]
- ◯ [4, 3, 2, 1, 0]
- ◯ [0, 1, 3, 2, 4]

4 **d[a][b] is the distance in km between places a and b; 0 is the school. Write nearest neighbour: keep adding the closest place not yet in route until all 5 are in. The program prints the route, its length (with the drive back) and how far the first village rides.**

```
d = [[0, 7, 1, 5, 2],
     [7, 0, 5, 2, 8],
     [1, 5, 0, 4, 3],
     [5, 2, 4, 0, 6],
     [2, 8, 3, 6, 0]]
route = [0]
# add the closest place not yet in route,
# until all 5 places are in

back = route[1:] + [0]
legs = zip(route, back)
km = sum(d[a][b] for a, b in legs)
print(route, km, km - d[0][route[1]])
```

Write your program:

5 Put the lines in order: a function that returns a route's length, including the drive back to the start.

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn't belong.

- [ ] `return km`
- [ ] `b = route[(i + 1) % len(route)]`
- [ ] `b = route[i + 1]`
- [ ] `for i in range(len(route)):`
- [ ] `def length(route):`
- [ ] `km += d[route[i]][b]`
- [ ] `km = 0`

✎ **Think of a route you take often. Is it the shortest, the safest or the one that suits others? What would you change, and who would that affect?**

Check your answers

The bus route

1 **The one where nobody rides more than 30 minutes**

When two plans cost the same, a second goal can decide between them.

2 **True**

Replacing the two crossing roads by the two that don't cross (reversing the stretch between them) is always shorter on a flat map, like the two diagonals of a four-sided shape against two of its opposite sides.

3 **[0, 3, 2, 1, 4]**

route[1:4] is [1, 2, 3]; reversed it is [3, 2, 1].

4

```
d = [[0, 7, 1, 5, 2],
     [7, 0, 5, 2, 8],
     [1, 5, 0, 4, 3],
     [5, 2, 4, 0, 6],
     [2, 8, 3, 6, 0]]
route = [0]
while len(route) < 5:
    last = d[route[-1]]
    left = [v for v in range(5)
            if v not in route]
    near = min(left, key=lambda v: last[v])
    route.append(near)
back = route[1:] + [0]
legs = zip(route, back)
km = sum(d[a][b] for a, b in legs)
print(route, km, km - d[0][route[1]])
```

Nearest neighbour goes 0 → 2 → 4 → 3 → 1 and back: 19 km, and the first village (2) rides 18 of them because it is picked up after only 1 km.

5 **Top to bottom: 6, 4, X, 3, 1, 5, 2**

```
def length(route):
    km = 0
    for i in range(len(route)):
        b = route[(i + 1) % len(route)]
        km += d[route[i]][b]
    return km
```

The % len(route) makes the last stop's next place the first one, so the drive back is counted.